

SECTION 1: BASIC FORTRAN

Lecture 1: Introduction and Background

FORTRAN (Formula Translator) developed by IBM in the 1950's; released in 1957 as a language for scientific computing.

Has gone through many versions.

Oldest you are likely to see is FORTRAN 77, in legacy code.

Fortran 90 was a major upgrade. New features included:

- Free-form source input
- Ability to operate on arrays as a whole
- Modules
- Derived data types
- Dynamic memory allocation
- Structured looping constructs
- Select ... Case construct

Fortran 95 was a minor upgrade

Fortran 2003 was another major upgrade which included object-oriented programming support.

Fortran 2008 was a minor upgrade.

Fortran 2018 is the latest release.

What is Fortran primarily used for today?

Large-scale scientific computing problems. Also very effective for day-to-day programs students might write, for instance, in a college engineering or mathematics class on numerical methods.

Easy to learn. Easy to code. Easy to debug.

Very fast code.

Ignore the endless on-line debates between programmers about which language is best. Likely you are not a computer scientist or professional programmer, but a user that needs a powerful, easy-to-learn scientific programming language *that has stood the test of time*.

By the end of this course, although you will not be at the level of a professional programmer, you should be able to write effective Fortran programs. We will concentrate on the Fortran 90/95 standard; however you will also be prepared to delve further into the more advanced features of the language should you so desire.

A Big-Picture Approach to Writing Your Fortran Codes

- Utilize a top-down, or structured, approach to programming.
- Clearly define your problem at the highest level.
- Break program into distinct components that can be tested separately and re-used in other programs/projects (utilize modules, subroutines, and functions). For instance, a routine to solve a set of linear equations.
- Thoroughly test your final code and look for ways in which users can “break” the code.

Where can you download a free Fortran compiler?

1) g95 compiler:

<http://www.g95.org/index.shtml>

or

<https://www.fortran.com/the-fortran-company-homepage/whats-new/g95-windows-download/>

2) gfortran compiler:

<https://gcc.gnu.org/wiki/GFortranBinaries>

3) gfortran compiler (ease of installation including openmp support, but much older release):

<https://sourceforge.net/projects/tdm-gcc/>

if you install from tdm-gcc, when you get to the components to install, be sure to open the gcc “tree” to add fortran and openmp

Compiling:

```
g95 [options] code.f90
```

or

```
gfortran [options] code.f90
```

Some useful g95 compiler options:

- c Compile and assemble, but do not link, produces object file
- o <file> Place the executable file into <file>
- d8 set the default real and integer kinds to double precision
- i8 Set kinds of integers without kind specifications to double default precision
- r8 Set kinds of reals without kind specifications to double default precision
- fbounds-check Checks array bounds at run time
- O[level] Optimization where “level” is typically 1, 2, or 3

Some useful gfortran compiler options:

- fopenmp Enables the OpenMP extensions.
- c Compile and assemble, but do not link, produces object file
- o <file> Place the executable file into <file>
- fdefault-integer-8 Set the default integer and logical types to double precision
- fdefault-real-8 Set the default real type to double precision
- fcheck=bounds Check array bounds at runtime
- fcheck=all Enable all checks
- O[level] Optimization where “level” is typically 1, 2, or 3

Lecture 2: Program Composition

1) Declaration Section

Non-executable statements

1st is to define program name (optional)

Define types of variables used in program

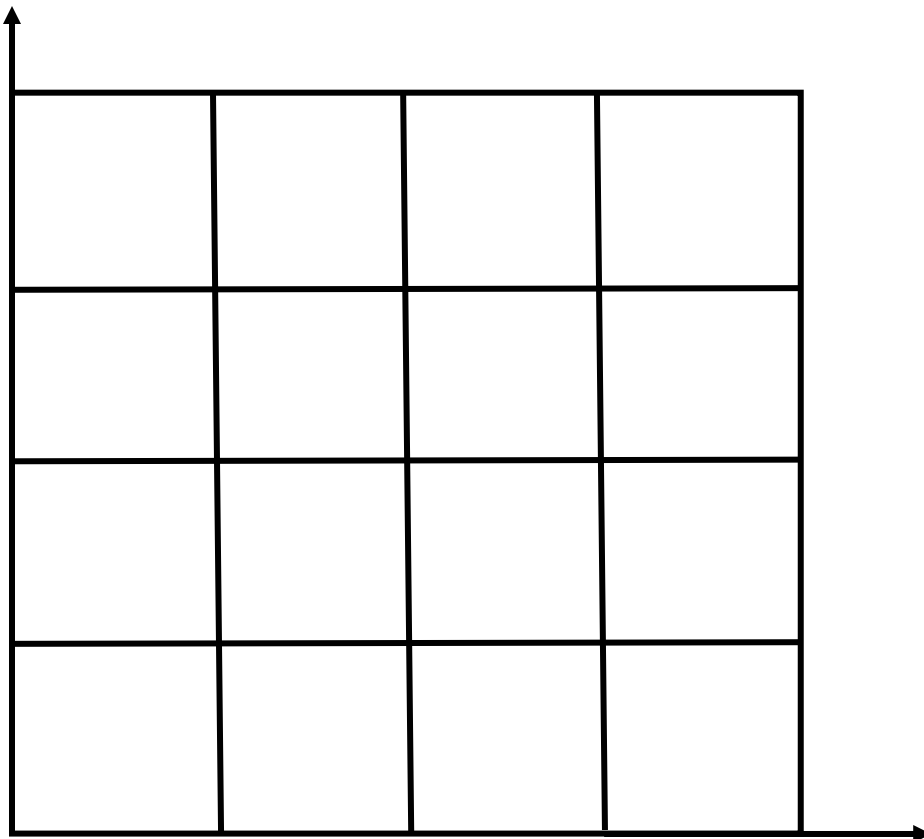
2) Execution Section

Consists of executable statements (usually bulk of code)

3) Termination Section

STOP (optional) and END PROGRAM statements

NEXT SLIDE; EXAMPLE CODE TO COMPUTE TEMPERATURE DISTRIBUTION
OVER A PLATE



Example Code to Compute Temperature Distribution in a Plate

! DECLARATION SECTION

```
PROGRAM EXAMPLE

IMPLICIT NONE

INTEGER, PARAMETER :: NX=3,NY=3

INTEGER :: i,j, outeriterations, &
           number_of_iterations=100

REAL, DIMENSION(0:NX+1,0:NY+1) :: t
```

! EXECUTION SECTION

```
!   INITIALIZE TEMPERATURE ARRAY t
t=0.0

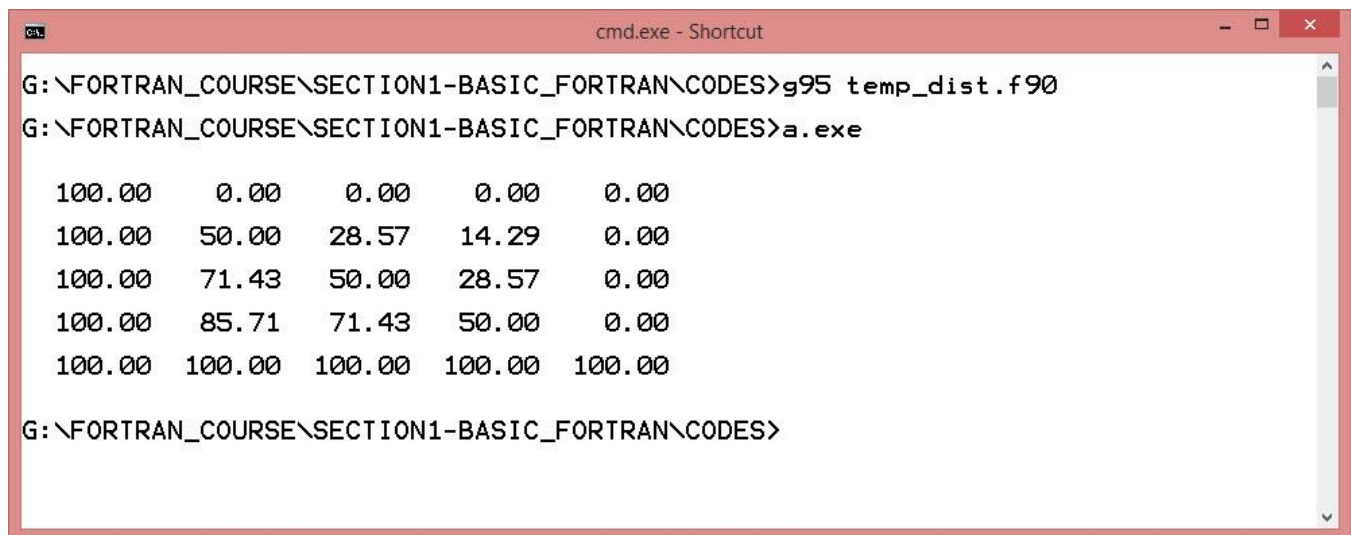
!   ADD BOUNDARY CONDITIONS
t(:,0)=100.;  t(0,:)=100.

!   ITERATE OVER DOMAIN
DO outeriterations=1,number_of_iterations
  DO i=1,NX
    DO j=1,NY
      t(i,j)=0.25*(t(i+1,j)+ &
                  t(i-1,j)+t(i,j+1)+t(i,j-1))
    END DO
  END DO
END DO
```

```
!      WRITE RESULTS TO SCREEN
      DO j=NY+1,0,-1
          WRITE(*,100) ( t(i,j),i=0,NX+1)
      END DO
100    FORMAT(20F6.2)
```

```
!      TERMINATION SECTION
      STOP                      ! OPTIONAL
      END PROGRAM EXAMPLE
```

PROGRAM OUTPUT



```
cmd.exe - Shortcut
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>g95 temp_dist.f90
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>a.exe

 100.00    0.00    0.00    0.00    0.00
 100.00   50.00   28.57   14.29    0.00
 100.00   71.43   50.00   28.57    0.00
 100.00   85.71   71.43   50.00    0.00
 100.00  100.00  100.00  100.00  100.00

G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>
```


Lecture 3: Data Types: Constants and Variables

Constants and Variables reserve locations in memory.

Variable names may be up to 31 characters long in Fortran90 (63 for Fortran2003)

Use meaningful names in your program.

Several types of constants and variables:

Integers

Real

3.1415

1.002E-03 (note decimal point required in mantissa)

Character

Character constant is a string of characters between single or double quotes

“A first course in Fortran” or ‘A first course in Fortran’

Declaring the “type” of variable

Consider the variable named: relaxation_factor

By default this is of type real

What about the variable: number_of_iterations

By default this is of type integer

In fact, variables beginning with letters a-h and o-z are real by default and those beginning with i-n are integer.

However, you should explicitly declare the variable type in your program declaration section.

Look at our example program declaration section:

```
PROGRAM EXAMPLE  
  
IMPLICIT NONE  
  
INTEGER, PARAMETER :: NX=3,NY=3  
  
INTEGER :: i,j, outeriterations, &  
           number_of_iterations=100  
  
REAL, DIMENSION(0:NX+1,0:NY+1) :: t
```

Character Type	Allowable Values
Upper and Lower Case Letters	a-z, A-Z
Digits	0-9
Underscore Character	_
Arithmetic Symbols	+ - * / **
Other Symbols	\$ () = . , ! % & " ' ? > : <

- There are a total of 85 valid symbols and a blank space.
- Fortran does not distinguish between upper and lower case letters, unlike some other languages.

Lecture 4: Assignment Statements and Operators

For instance,

```
number_iterations=100
```

or

```
iteration=iteration+1
```

Operators

+

-

/

*

**

Operators cannot appear directly adjacent to each other

i.e., $a/-5.$ is not legal

instead use $a/(-5.)$

Integer Arithmetic

$4/4 \rightarrow 1$

$4*4 \rightarrow 16$

$4/5 \rightarrow 0$

$$5/4 \rightarrow 1$$

$$30/8 \rightarrow 3$$

Real Arithmetic

$$4./5. \rightarrow 0.8$$

$$5./4. \rightarrow 1.25$$

$$30./8. \rightarrow 3.75$$

$$4./7. \rightarrow 0.571428...$$

Order of operations

- 1) operations within parenthesis
- 2) exponentials, in order from right to left
- 3) multiply and divide, in order from left to right
- 4) add and subtract, in order from left to right

Examples

$$1.+1./2. \rightarrow 1.5$$

$$(1.+1.)/2. \rightarrow 1.$$

$$9**2/4 \rightarrow 20.25$$

$$9**(2./4.) \rightarrow 3.$$

$$2**2**3 \rightarrow 256$$

$$(2**2)**3 \rightarrow 64$$

$$(6+3)*2*(2+2)/2**2 \rightarrow 18$$

$$3.*4./2.**3 \rightarrow 1.5$$

$$3.*(4./2.**3) \rightarrow 1.5$$

$$(3.*4.)/2.**3 \rightarrow 1.5$$

Mixed Mode Arithmetic

Operation between 2 real numbers results in a real number

Operation between 2 integer numbers results in an integer

What about an operation between a real and an integer?

The integer is converted to real

Examples

$$1/4. \rightarrow 0.25$$

$$9.** (2/4) \rightarrow 1.$$

$$9.** (2./4) \rightarrow 3.$$

$$9** (2./4) \rightarrow 3.$$

Lecture 5: Intrinsic Functions

For example:

Use Intrinsic Functions to Control Conversions

Examples

REAL(1)=1. Convert integer to real

INT(1.6)=1 Convert real to integer

NINT(1.6)=2 Convert real to integer

CEILING(1.6)=2 Convert real to integer

FLOOR(1.6)=1 Convert real to integer

Other Common Functions

ABS(X)

SQRT(X)

SIN(X)

COS(X)

TAN(X)

ASIN(X)

ACOS(X)

ATAN(X)

EXP(X)

LOG(X)

MAX(A,B,C,...) MIN(A,B,C,...)

DOT_PRODUCT(vector1, vector2)

Lecture 6: Basic Input/Output

List-Directed or Formatted

List-Directed is much simpler and what we will discuss in this Section/Lecture.

Form for OUTPUT

PRINT*, output-list (very useful for debugging)

Or

WRITE(*,*) output-list

Form for INPUT

READ *, input-list

Or

READ(*,*) input-list

Generally reads data from a keyboard.

Program should prompt user beforehand.

Let's look at a simple example showing list-directed input/output:

```

!      List-Directed input/output
PROGRAM input_output
IMPLICIT NONE
INTEGER :: num_students,num1,num2

PRINT*; PRINT*
PRINT*, 'Enter the number of students'
READ*, num_students
PRINT*
PRINT*, 'The number of students is: &', num_students

!      OR

PRINT*; PRINT*
WRITE(*,*) 'enter 2 numbers'
READ(*,*) num1,num2
PRINT*, 'number1 = ', num1, '    number2 = ', num2

END PROGRAM input_output

```



```
cmd.exe - Shortcut
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>a.exe
Enter the number of students:
4
The number of students is: 4
enter 2 numbers
2 6
number1 = 2    number2 = 6
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>a.exe
Enter the number of students:
7
The number of students is: 7
enter 2 numbers
2 6
number1 = 2    number2 = 6
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>a.exe
Enter the number of students:
7
The number of students is: 7
enter 2 numbers
4,9
number1 = 4    number2 = 9
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>
```

Lecture 7: Section 1 Example Code

```
!      Program to Compute Monthly Compounded Interest
PROGRAM compound_interest
IMPLICIT NONE
INTEGER, PARAMETER :: n=12    !compounded monthly
INTEGER :: years
REAL :: total, principle, interest_rate

! DEFINITIONS
!interest_rate=input as annual interest rate in %; then
converted to decimal
!years=time in years for interest to accululate
!n=number of times interest is compounded per unit time (a
year); a constant
!principle=initial investment amount
!total=future value of investment after interest payments

PRINT*
PRINT*, 'COMPOUNDED MONTHLY INTEREST CALCULATOR'
PRINT*
PRINT*, 'Enter the initial investment amount in $'
READ(*,*)principle

WRITE(*,*) 'Enter the number of years for the investment'
READ *,years
PRINT*, 'Enter the percent interest rate'
```

```

READ(*,*)interest_rate

interest_rate=interest_rate/100.      !convert rate to a
decimal

PRINT*,'The initial principle is:',principle
PRINT*,'The interest rate is: ',interest_rate*100,'% '
!total value of investment after "years"
total=principle*(1.0 + interest_rate/n)**(n*years)
PRINT*,'The value of your investment after',years,'years is
$',total

END PROGRAM compound_interest

```

```

cmd.exe - Shortcut

G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>gfortran compound_interest.f90
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>a.exe
  COMPOUNDED MONTHLY INTEREST CALCULATOR
Enter the initial investment amount in $
1000
Enter the number of years for the investment
10
Enter the percent interest rate
6
The initial principle is: 1000.000000
The interest rate is: 6.000000000 %
The value of your investment after 10 years is $ 1819.39954
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>a.exe
  COMPOUNDED MONTHLY INTEREST CALCULATOR
Enter the initial investment amount in $
1000
Enter the number of years for the investment
10.
At line 22 of file compound_interest.f90 (unit = 5, file = 'stdin')
Fortran runtime error: Bad integer for item 1 in list input
G:\FORTRAN_COURSE\SECTION1-BASIC_FORTRAN\CODES>

```

Note broken output when entering real value for “number of years for the investment.”